

PyPI-SCADA — Sitrep 2 Speaker Notes

Slide 1 — Title

- Frostbyte Cybersecurity Research Laboratory, Reykjavik University

Slide 2 — Supply Chain Example

- Farms → tractors → warehouse → trucks → retail
- If one farm is compromised, every supermarket downstream gets poisoned product

Slide 3 — PyPI Supply Chain

- Same structure: developers upload via Twine/CI/CD → PyPI → pip → production
- Bottom-left guy is a malicious actor — blends right in

Slide 4 — PyPI Today

- 740K+ projects, 103B+ downloads last month
- Google, Meta, Netflix, Amazon, JPMorgan all depend on this
- One poisoned package can cascade into thousands of production systems

Slide 5 — Fruit Poisoning (visual)

- Red farm = compromised maintainer, poison flows through trusted channels

Slide 6 — Fruit Poisoning (details)

- **num2words**: 23M+ total downloads, used for converting numbers to words in invoicing, finance, localization — attacker used a forward proxy to intercept MFA tokens in real time, injected an obfuscated DLL loader into `__init__.py` — every app that auto-updated got compromised silently
- **ultralytics**: 260M+ total downloads, one of the most popular computer vision libraries — attacker didn't even need credentials, exploited a GitHub Actions script injection via malicious PR — the XMRig cryptominer was baked into the published wheel while the source repo stayed clean, so nobody reviewing the code on GitHub would see it

Slide 7 — Typosquatting (visual)

- Skull = malicious actor, registers lookalike package names

Slide 8 — Typosquatting (details)

- **colourama** (targeting colorama, 600M+ downloads): base64-encoded clipboard hijacker — silently swapped cryptocurrency wallet addresses when you copy-pasted, victims sent money directly to the attacker without realizing
- **nmap-python** (targeting python-nmap): combosquatting, reversed word order — embedded trojan targeting anyone scanning networks, ironic because the people installing it are likely security professionals
- **termncolor** (targeting termcolor): multi-stage — imported a rogue package called **colorinal** that triggered DLL side-loading for persistence, designed to survive reboots

Slide 9 — Dependency Confusion (visual)

- Blue = private registry, orange = public, red = malicious
- pip installs highest version number regardless of source

Slide 10 — Dependency Confusion (details)

- **torchtriton**: PyTorch's nightly builds depended on an internal package — attacker registered **torchtriton** v2.0.0 on public PyPI with artificially high version — automated build systems silently pulled the malicious one instead — payload was an ELF binary that exfiltrated SSH keys and system data via DNS tunneling, meaning it bypassed most firewalls since DNS traffic looks normal
- **totallysafe**: early proof-of-concept, arbitrary code execution in setup script — showed that just running `pip install` is enough to get owned, no import needed

Slide 11 — Current Landscape of Detection

- PyPI is reactive: post-incident + community reporting
- Hardened with MFA, domain credential freezing, some static analysis, trust chains
- But attacks still get through — that's why this project exists

Slide 12 — What Will PyPI-SCADA Do?

- Not a silver bullet — building the infrastructure to benchmark
- SAST vs LLMs using real payloads from Backstabber's Knife Collection

Slide 13 — Evaluation Methodology

- **SAST**: Bandit (AST) + Semgrep (pattern) + heuristic pre-filter
- **LLM**: 6 models × 3 prompt strategies (zero-shot, few-shot, role-based)
- **Agentic**: Claude Code + Codex, multi-step autonomous

- All three get identical input — fair comparison

Slide 14 — Malicious Sample Set

- 9 packages, 4 attack vectors
- Each row: package → vector → target → payload mechanism

Slide 15 — Design Pivot

- OLD: diff N vs N+1 — fails for typosquatting (no v0), dependency confusion (malicious from v1), version dilution (clean v2.1 hides malicious v2.0)
- NEW: extract setup.py, __init__.py, pyproject.toml + their imports
- Heuristic pre-filter: base64, network imports, exec/eval, bundled binaries

Slide 16 — Timeline: Completed

- Phase I + II done, diffing engine pivoted, sample curation done

Slide 17 — Timeline: Phase III

- Dataset finalization Mar 16–Apr 5
- Pipeline integration Apr 6–12
- Execution Apr 13–26 during exam window

Slide 18 — Timeline: Phase IV

- Sitrep 3 dress rehearsal Apr 27–May 3
- Skemman submission May 4–17
- Public defense May 18–22

Slide 19 — Risk Assessment

- Highest: Scope Creep (12) and False Positives (12)
- Risk diary: pivot triggered scope creep, contained by keeping PoC scope

Slide 20 — Design of PyPI-SCADA

- Three components: Injector → Simulator → Detection-Analyzer

Slide 21 — PyPI Injector

- Shell controller uses twine, SQL API serves samples, YAML config for prompts

Slide 22 — PyPI Simulator

- Flask on localhost:8080, PEP 503 compliant
- main.py = Flask app, simple.py = Simple API endpoints

Slide 23 — PyPI Analyzer

- entry_extractor.py replaces diff parser
- heuristic_filter.py = triage layer
- detection_controller.py orchestrates StaticDetectorAdapter + LLMDetectorAdapter
- Result DB stores RUN_IDs, ground truth never enters the analyzer

Slide 24 — Worked Hours

- 146h of 380h (38%), 234h remaining, 9 weeks — fits schedule

Slide 25 — Prototype Demo (1)

- Flask running, twine uploads work, Simple Index lists packages

Slide 26 — Prototype Demo (2)

- Version list, JSON API, pip install from local server works

Slide 27 — Results and Future Works

- Statistical benchmarks in final thesis
- Future work: production analysis sidecar for PyPI

Slide 28 — Schedule Status

- Phase I + II done, Phase III starting now, on track

Slides 29–31 — Feedback & Questions

- Use the flowchart slide (31) as closing visual